基于 Rust 的 openEuler 内核模块开发与动态加载机制实验指导书

翻转课堂任务指导书　第六组

小组成员 隋元皓 李志杰 张创举 赵虹尊 刘晓航 丁厚凯

本实验在 openEuler 虚拟机中完成四组 C/Rust 内核模块对照实验，从 Hello 模块开始，逐步实现 /proc 数值读写、命令解析和字符设备交互。学习者应能搭建支持 Rust 的内核开发环境，完成模块编译、动态加载与卸载，并结合源码解释用户态到内核态的调用过程。

内容安排 一、实验环境构建　二、实验设计　三、实验流程

一 实验环境构建

1 实验平台与准备条件

项目	配置或要求
虚拟化平台	VMware；在虚拟机内执行全部内核实验
操作系统	openEuler 24.03 LTS-SP4，x86_64
实验内核	启用 Rust 支持的 Linux 6.6；视频运行版本为 6.6.0-rustlab
工具链	汇报截图为 Clang 17.0.6、Rust 1.73.0；使用匹配的构建树
开发界面	Windows 上的 VS Code 与 Remote - SSH；Linux 终端执行命令
建议资源	4 核 CPU、8 GB 内存、至少 40 GB 可用磁盘；编译前检查空间

```
[root@localhost os-kernel-labs]# cat /etc/os-release
NAME="openEuler"
VERSION="24.03 (LTS-SP4)"
ID="openEuler"
VERSION_ID="24.03"
PRETTY_NAME="openEuler 24.03 (LTS-SP4)"
ANSI_COLOR="0;31"
```

图 1 操作系统发行版本核验 对应 PPT 第 4 页

完成条件 可以登录虚拟机，具有 sudo 权限，能够访问软件仓库。已配好实验虚拟机的同学可直接核验第 5 节，再进入源码准备。

2 创建虚拟机并连接开发终端

步骤 1 在 VMware 中新建 Linux 虚拟机，使用 openEuler 24.03 LTS-SP4 x86_64 安装镜像完成安装。网络可选 NAT，确认网卡处于已连接状态；创建普通实验用户并授予 sudo 权限。首次编译内核前创建虚拟机快照。

步骤 2 在虚拟机终端确认网络与 SSH。以下命令均在 openEuler 中运行；网卡名、连接名和 IP 地址以实际输出为准。

```
cat /etc/os-release
uname -r
uname -m
ip -br addr
sudo dnf install -y openssh-server
sudo systemctl enable --now sshd
sudo systemctl status sshd --no-pager
```

步骤 3 若启用了 firewalld，执行 sudo firewall-cmd --permanent --add-service=ssh，再执行 sudo firewall-cmd --reload。随后在 Windows 终端使用 ssh 用户名@虚拟机 IP 测试登录。

步骤 4 VS Code 安装 Remote - SSH 扩展，添加同一 SSH 目标并连接。打开虚拟机中的 ~/os-kernel-labs，确认集成终端运行的是 Linux 命令。截图中的 192.168.75.129 是原演示地址，复现时使用当前虚拟机地址。

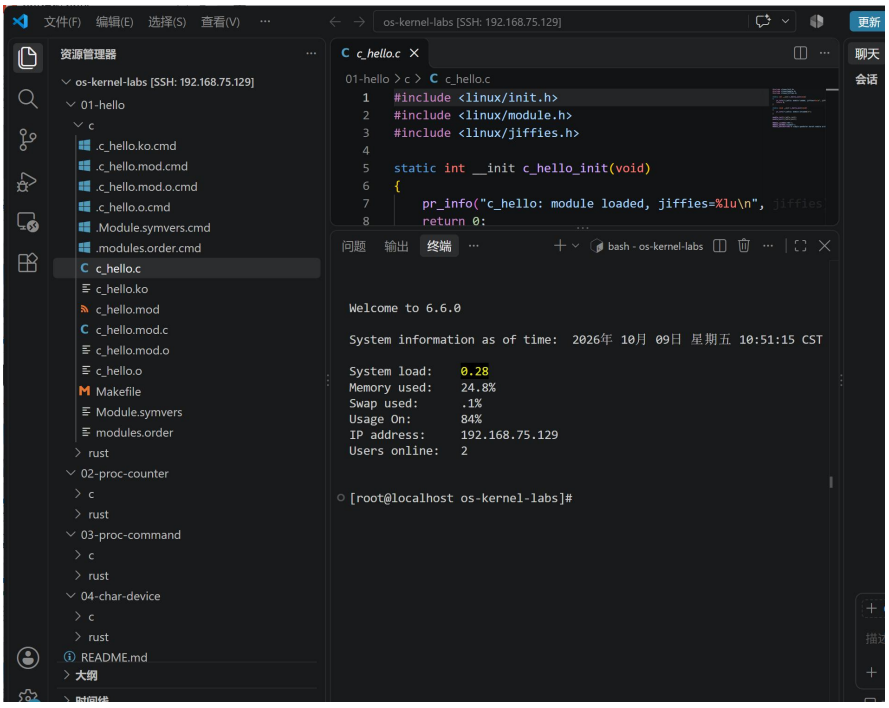


图 2 VS Code 通过 SSH 打开实验工程 截取自 PPT 第 3 页

验收 SSH 登录成功，终端执行 uname -r 能得到虚拟机内核版本。重启后连接失败时，先在 VMware 控制台检查 IP 是否变化，再检查 sshd 和 NetworkManager。

3 安装构建工具并固定工具链

步骤 1 在虚拟机安装构建依赖。软件包名可能随仓库变化，安装失败时先用 dnf search 查询，不要直接跳过缺失依赖。Clang 主版本应与内核构建环境一致。

```
sudo dnf install -y make gcc gcc-c++ git bc bison flex \
  openssl-devel elfutils-libelf-devel ncurses-devel \
  dwarves clang llvm lld python3 perl curl xz cpio unzip binutils file kmod
clang --version
ld.lld --version
df -h "$HOME" /boot
```

步骤 2 若没有 rustup，从 Rust 官方渠道安装 rustup；在实验用户下安装源码 Makefile 指定的 Rust 1.73.0，并安装对应 rust-src。不要只安装最新版 Rust 后直接编译旧内核。

```
# 仅在尚未安装 rustup 时执行前两行
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs -o /tmp/rustup-init.sh
sh /tmp/rustup-init.sh --default-toolchain none
. "$HOME/.cargo/env"
rustup toolchain install 1.73.0 --profile minimal
rustup component add rust-src --toolchain 1.73.0
rustup run 1.73.0 rustc --version
bindgen --version
```

步骤 3 准备与实验内核配套的 bindgen-cli。若未安装或版本不匹配，在内核源码树运行 make LLVM=1 rustavailable，根据输出要求安装相应版本；可用 cargo +1.73.0 install --locked bindgen-cli --version 版本号。bindgen 还需要可用的 libclang，缺失时安装对应开发包。

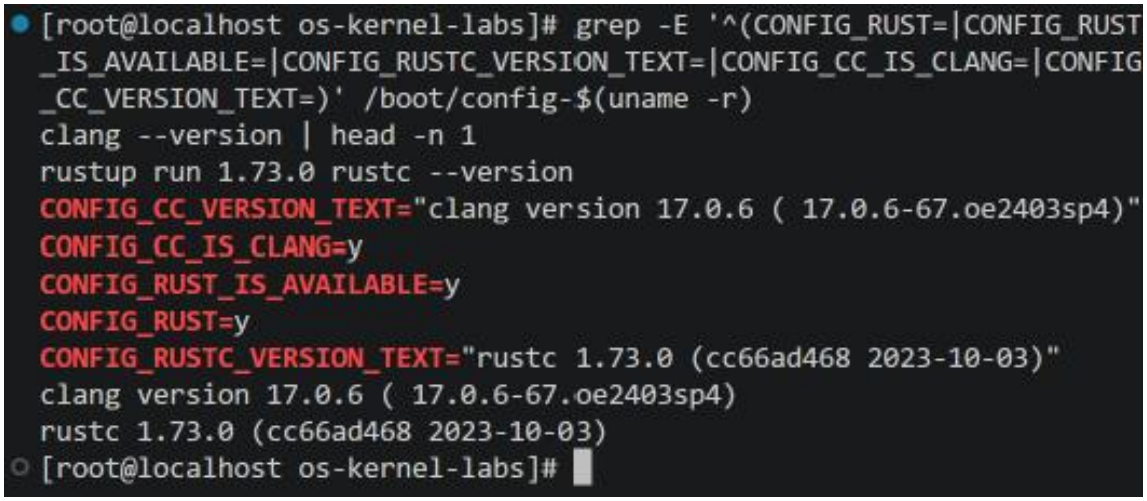


图 3 Rust 内核配置与编译器版本 对应 PPT 第 4 页

验收 rustc 输出 1.73.0；Clang 与目标内核的编译器一致；bindgen、rust-src、libclang 均可被构建系统找到。截图所示配置是目标状态，不等同于发行版默认状态。

4 配置并编译支持 Rust 的 Linux 内核

步骤 1 准备 Linux 6.6 内核源码。优先使用小组实验虚拟机保留的已配置源码树；既有构建记录指向 ~/kernel-build/linux-rust。新环境可从 kernel.org 获取 Linux 6.6 源码作为起点，随后按 rustavailable 的检查结果处理工具链兼容性。

```
mkdir -p ~/kernel-build
cd ~/kernel-build
# 新搭环境需要下载源码；已有配套源码树时不重复下载
curl -LO https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.6.tar.xz
tar -xf linux-6.6.tar.xz
cd linux-6.6
export RUSTUP_TOOLCHAIN=1.73.0
make LLVM=1 rustavailable
```

步骤 2 以虚拟机当前内核配置为起点生成配置。发行版配置可能包含上游内核不支持的选项，需通过 olddefconfig 归一化；若提示证书文件缺失，按实际构建日志处理证书路径。

```
cp /boot/config-$(uname -r) .config
make LLVM=1 olddefconfig
make LLVM=1 menuconfig
```

步骤 3 在配置界面启用模块支持与 Rust 支持，保留虚拟机启动所需的磁盘、文件系统、网卡和串口等驱动。CONFIG_RUST_IS_AVAILABLE 是工具检测结果，不能靠手工设为 y 代替检查。

配置项	目标与核验方式
CONFIG_MODULES	y，允许加载 .ko 模块
CONFIG_MODULE_UNLOAD	y，支持 rmmod 卸载
CONFIG_RUST	y：依赖工具链和架构配置满足 Rust 条件
CONFIG_RUST_IS_AVAILABLE	y：make rustavailable 检查通过
CONFIG_PROC_FS	y：用于实验二、三的 /proc 接口

步骤 4 保存配置后执行 make LLVM=1 -j$(nproc) 完整构建。内存不足时减少并行任务数，例如 -j2。检查最终生成 arch/x86/boot/bzImage、vmlinux 和 Module.symvers。

验收 rustavailable 通过，完整内核构建成功。仅运行 modules_prepare 不能替代完整构建，尤其不能保证 Rust 库产物和符号版本文件齐全。

5 安装实验内核并核验构建树

步骤 1 在刚完成构建的源码目录安装内核及模块，保留原内核启动项。以下操作在虚拟机内进行。

```
export RUSTUP_TOOLCHAIN=1.73.0
make -s kernelrelease
sudo make LLVM=1 modules_install
sudo make LLVM=1 install
# 记下 kernelrelease 的输出，重启时选择这个内核
sudo reboot
```

步骤 2 在 VMware 控制台的引导菜单选择新内核。若安装流程未生成 initramfs 或引导项，依据 openEuler 的引导配置使用 dracut 和 grubby 补齐，确认 /boot 中的内核与 initramfs 成对存在后再启动。

步骤 3 重新 SSH 连接，核验运行内核、配置和 build 链接。若构建时添加了 -rustlab 后缀，uname -r 应包含该后缀；不硬编码版本字符串。

```
uname -r
cat /proc/version
grep -E '^(CONFIG_MODULES|CONFIG_MODULE_UNLOAD|CONFIG_RUST|CONFIG_RUST_IS_AVAILABLE)=' \
  /boot/config-$(uname -r)
readlink -f /lib/modules/$(uname -r)/build
test -s /lib/modules/$(uname -r)/build/Module.symvers && echo symvers_OK
ls /lib/modules/$(uname -r)/build/rust/
make -s -C /lib/modules/$(uname -r)/build kernelrelease
```

步骤 4 build 应指向本次生成运行内核的完整构建树。若链接缺失，先确认正确源码目录，在 /lib/modules/$(uname -r)/ 下建立指向该目录的 build 链接；不要把不同内核的源码树强行连过来。

核验项目	通过标准
运行版本与构建版本	uname -r 与 make -s kernelrelease 完全一致
Rust 支持	CONFIG_RUST=y；rust/ 下保留所需编译产物
模块符号版本	Module.symvers 来自同一完整内核构建
编译器	与该内核匹配的 LLVM/Clang 与 Rust 工具链
可恢复性	原内核启动项与虚拟机快照可用

验收 上述条件同时满足后再编译实验模块。仅版本号相同仍不足以证明 ABI、内核配置和符号版本匹配。

6 准备源码并完成首次编译

步骤 1 将本指导书配套的“配套源码.zip”上传到虚拟机用户主目录并解压。包内顶层目录为 os-kernel-labs，含 01 至 08 子目录。配套包保留原始 C/Rust 主源码，并补入三个既有桥接文件，不包含 .o、.ko 等旧构建产物。

```
# 以下路径为推荐的虚拟机工作目录
cd "$HOME"
unzip 配套源码.zip -d "$HOME"
export LAB="$HOME/os-kernel-labs"
cd "$LAB"
ls
```

步骤 2 若直接使用原“源代码干净”目录，必须先补齐桥接源码。下表中的来源文件均在同级“源代码及其编译产物”内，对应复制到同名实验子目录。

实验子目录	需要补入的文件	作用
06-rust-proc-counter	proc_shim.c	注册 /proc，用户缓冲区拷贝，调用 Rust
07-rust-proc-command	proc_shim.c	命令计数器的 proc_ops 适配
08-rust-char-device	char_shim.c	miscdevice 注册与文件操作适配

步骤 3 先用 Hello 模块验证工具链。所有实验统一使用与目标内核匹配的 LLVM 构建方式；Rust Makefile 已固定 RUSTUP_TOOLCHAIN=1.73.0。

```
cd "$LAB/01-c-hello"
make LLVM=1 clean
make LLVM=1
modinfo c_hello.ko | head
cd "$LAB/05-rust-hello"
make clean
make
modinfo -F vermagic rust_hello.ko
uname -r
```

步骤 4 检查 .ko 文件生成，确认 vermagic 中的内核版本字段与当前内核一致，再执行 sudo insmod rust_hello.ko、sudo dmesg | tail -n 10 和 sudo rmmod rust_hello。应分别看到 loaded 与 unloaded 日志。

环境阶段验收 SSH 可用、内核 Rust 支持开启、构建树匹配、八个实验目录齐全、三个 shim 文件齐全、C/Rust Hello 均能编译并正常加载卸载。

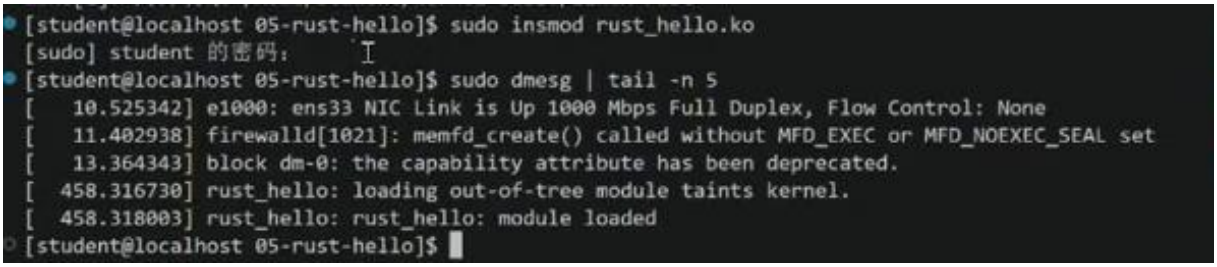


图 4 Hello 加载日志 截取自实验 1 演示视频约 30 秒处

二 实验设计

实验主线为“环境核验 → Hello 生命周期 → /proc 数值交互 → /proc 命令解析 → 字符设备交互 → 模块并存验证 → 卸载清理”。其中环境构建、用户态与内核态交互、命令解析和字符设备行为是实验报告需要重点分析的部分。

1 四组对照实验

实验	C 与 Rust 目录	核心任务与观测
一 Hello	01-c-hello 05-rust-hello	编译、insmod、rmmod、dmesg；理解 init、Module、Drop
二 数值计数器	02-c-proc-counter 06-rust-proc-counter	通过 /proc 写入数字并读取状态；有效输入、非法输入、读写计数
三 命令计数器	03-c-proc-command 07-rust-proc-command	解析 inc、dec、reset 和整数；记录成功命令与无效输入
四 字符设备	04-c-char-device 08-rust-char-device	使用 /dev 节点完成 open/read/write/release；解释计数差异

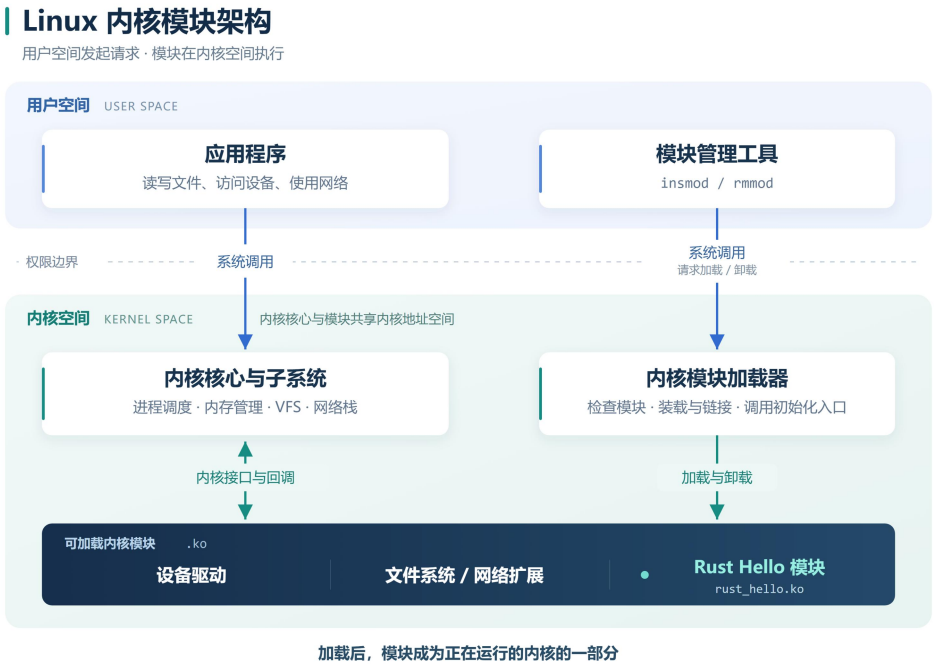


图 5 内核模块在 Linux 架构中的位置 对应 PPT 第 6 页

比较范围 两种语言都运行在同一内核态、使用同一模块加载机制。实验二、三比较相近业务；实验四的 C 版保存消息，Rust 版解析计数命令，应比较设备接口、数据边界与清理机制。

2 实验变量与验收设计

控制条件 同一虚拟机、运行内核和构建树；各实验开始前卸载旧模块再重新加载，使计数从初始状态开始。使用相同的短数字和短命令测试。每次 cat 都可能改变 read_count，应把“读取状态”本身记入操作记录。

验证目标	操作或输入	通过标准
生命周期	insmod → lsmod → rmmod	模块出现后消失，日志有加载与卸载记录
有效数值	42、-7	counter 与写入值一致；成功写入计数增加
无效数值	hello	写入返回错误，原 counter 不变；Rust invalid_count 增加
命令分发	12 → inc → dec → reset → -7	counter 依次为 12、13、12、0、-7
无效命令	bad	command_count 不增加，invalid_count 增加
设备生命周期	cat /dev/rust_lab_char	本次 open 已计入，本次 release 在读取后发生
并发增量	两路各执行 100 次 inc	无其他写者时最终 counter=200
模块并存	同时加载 Rust Hello 与字符设备	两者同时出现在 lsmod；卸载 Hello 后设备可继续读取
清理	rmmod 所有实验模块	对应 /proc 和 /dev 节点消失

3 翻转课堂组织与记录

课前 完成环境搭建与源码阅读，能够指出 init、卸载回调、读写回调和桥接文件所在位置。准备好已编译内核的虚拟机，课堂不等待完整内核编译。

课中 按四组实验依次演示。每组由操作者执行命令，观察者记录状态变化，讲解者结合源码说明回调路径与错误处理。具体人员分工由小组现场安排。

课后 整理环境核验截图、终端命令、关键输出与源码分析。对照预期解释差异，记录实际内核版本、工具链版本及重复实验的初始状态。

记录口径 非法输入发生在模块运行时，应观察 shell 错误提示和内核日志；这不是重新编译时报错。单次原子操作保证单变量更新，但多字段读取不构成一致的事务快照。

三 实验流程

1 通用准备与动态加载机制核验

先在 openEuler 终端设置 LAB，检查没有遗留实验模块，并确认本次使用的是新编译的 .ko。以下命令以 Rust Hello 为例。

```
export LAB="$HOME/os-kernel-labs"
uname -r
lsmod | grep -E '^(c_|rust_)'
cd "$LAB/05-rust-hello"
make clean
make
file rust_hello.ko
modinfo rust_hello.ko
readelf -h rust_hello.ko
readelf -S rust_hello.ko
readelf -r rust_hello.ko
nm -u rust_hello.ko
```

观察 .ko 的 ELF 头、节区和重定位项，查找代码段、数据段、.modinfo 与符号表。nm -u 中出现未定义的外部符号并不自动代表构建失败，部分符号在加载时由内核导出符号解析。

```
sudo insmod rust_hello.ko
lsmod | grep '^rust_hello'
sudo dmesg | tail -n 10
sudo rmmod rust_hello
lsmod | grep '^rust_hello'
sudo dmesg | tail -n 10
```

原理对应 PPT 第 8 至 11 页

insmod 通常通过 finit_module 或 init_module 请求内核装载；加载器检查模块、分配内核虚拟地址空间、解析可用导出符号并应用架构相关重定位，随后调用初始化入口。卸载时执行退出逻辑、注销接口并回收资源。

“地址 = 基址 + 偏移”可用于理解简单节内定位，但真实 ELF 重定位还取决于重定位类型、符号值与加数等。Module.symvers 是构建阶段的导出符号及版本信息文件，不是运行时直接查询地址的全局符号表。

验收 保存 readelf 重定位输出、modinfo 元数据，以及 loaded/unloaded 两条日志。rmmod 使用模块名，不带 .ko 后缀。

2 实验一 Hello 模块生命周期

目标 对应 PPT 第 16 至 19 页。理解内核模块没有用户程序式 main 入口，加载与卸载由内核回调触发；对照 C 的宏注册和 Rust 的 Module、Result、Drop。

```
cd "$LAB/01-c-hello"
make LLVM=1 clean && make LLVM=1
sudo insmod c_hello.ko
sudo dmesg | tail -n 5
sudo rmmod c_hello
sudo dmesg | tail -n 5
cd "$LAB/05-rust-hello"
make clean && make
sudo insmod rust_hello.ko
sudo dmesg | tail -n 5
sudo rmmod rust_hello
sudo dmesg | tail -n 5
```

关键源码对照

源码 01-c-hello/c_hello.c 第 5 至 17 行

```
static int __init c_hello_init(void)
{
    pr_info("c_hello: module loaded, jiffies=%lu\n", jiffies);
    return 0;
}

static void __exit c_hello_exit(void)
{
    pr_info("c_hello: module unloaded\n");
}

module_init(c_hello_init);
module_exit(c_hello_exit);
```

源码 05-rust-hello/rust_hello.rs 第 17 至 28 行

```
impl kernel::Module for RustHello {
    fn init(_module: &'static ThisModule) -> Result<Self> {
        pr_info!("rust_hello: module loaded\n");
        Ok(RustHello)
    }
}

impl Drop for RustHello {
    fn drop(&mut self) {
        pr_info!("rust_hello: module unloaded\n");
    }
}
```

预期 C 日志包括 c_hello: module loaded 与 module unloaded；Rust 日志包括 rust_hello: module loaded 与 module unloaded。C 额外打印 jiffies，数值随系统运行变化，不要求与演示截图相同。

分析与验收 指出 C 中 module_init/module_exit 如何绑定回调，Rust 的 Ok(RustHello) 如何表示初始化成功，以及模块对象销毁时 Drop 如何执行。资源注销函数仍需开发者正确实现，不能仅凭出现 Drop 就认定所有资源自动清理。

3 实验二 proc 数值计数器交互

目标 对应 PPT 第 20 至 23 页。通过 /proc 完成“用户输入 → 内核解析 → 状态更新 → 用户读取”，观察有效数字和非法字符串两条运行路径。

```
cd "$LAB/02-c-proc-counter"
make LLVM=1 clean && make LLVM=1
sudo insmod c_proc_counter.ko
cat /proc/os_lab_counter
printf '42\n' | sudo tee /proc/os_lab_counter > /dev/null
cat /proc/os_lab_counter
printf 'hello\n' | sudo tee /proc/os_lab_counter > /dev/null
cat /proc/os_lab_counter
sudo rmmod c_proc_counter

cd "$LAB/06-rust-proc-counter"
make clean && make
sudo insmod rust_proc_counter.ko
cat /proc/rust_lab_counter
printf '42\n' | sudo tee /proc/rust_lab_counter > /dev/null
cat /proc/rust_lab_counter
printf 'hello\n' | sudo tee /proc/rust_lab_counter > /dev/null
cat /proc/rust_lab_counter
```

阶段	C 版预期	Rust 版预期
刚加载后第一次 cat	counter=0，write_count=0	counter=0，write_count=0，invalid_count=0
写入 42 后读取	counter=42，write_count=1	counter=42，write_count=1
写入 hello 后读取	返回错误，counter 仍为 42；未设置 invalid_count 字段	返回错误，counter 仍为 42；invalid_count=1，write_count 仍为 1

注意 sudo tee 提权的是写操作，避免 sudo echo … > 文件 中重定向仍由普通 shell 执行的问题。tee 的标准输出已丢弃，读到的状态只来自 cat。

验收 再写入 -7 验证负数解析，保存状态输出和非法输入的错误提示。最后 sudo rmmod rust_proc_counter，并检查两个 /proc 节点均已移除。

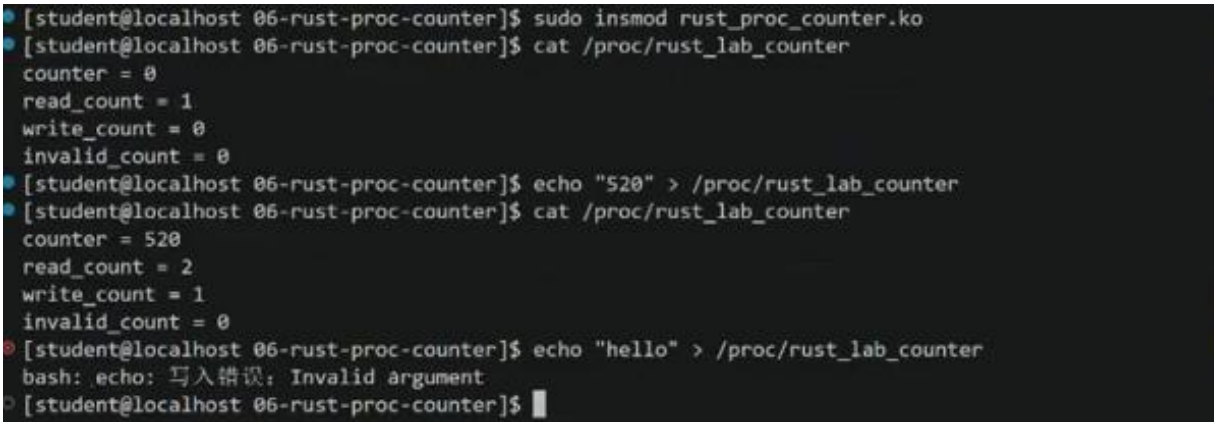


图 6 实验 2 视频约 45 秒处 原演示写入 520 和 hello 正文统一使用 42

4 实验二的回调路径与安全边界

调用路径 cat 打开 /proc 节点得到文件描述符，read 沿当前进程的 FD 表找到 struct file，再由 VFS/procfs 进入已注册读回调。Rust 版先进入 proc_shim.c，随后调用 Rust render 函数；格式化结果通过 simple_read_from_buffer 返回用户缓冲区。

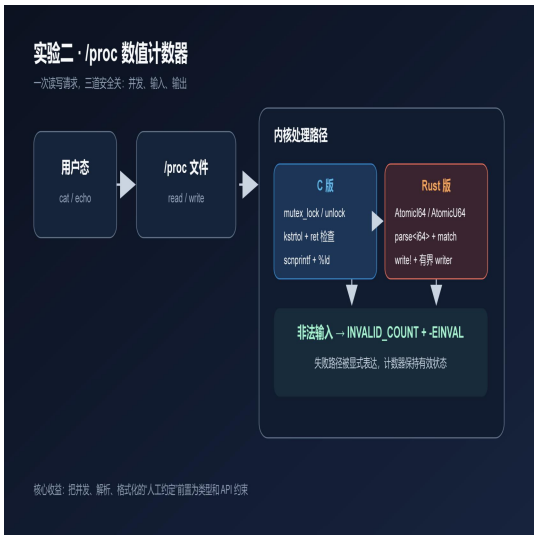


图 7 proc 计数器的 C 与 Rust 实现对照 对应 PPT 第 23 页

共享状态与解析结果

```
// Rust 数值解析的关键路径，节选并省略日志
match input.parse::<i64>() {
    Ok(value) => {
        COUNTER.store(value, Ordering::Relaxed);
        WRITE_COUNT.fetch_add(1, Ordering::Relaxed);
        len as c_long
    }
    Err(_) => {
        INVALID_COUNT.fetch_add(1, Ordering::Relaxed);
        EINVAL_NEG
    }
}
```

源码对应 C 文件中的 counter_write、counter_read、counter_proc_ops；Rust 文件中的 rust_proc_counter_store、rust_proc_counter_render、KernelBuf；桥接文件中的 rust_counter_write 与 rust_counter_read。

C 使用互斥锁保护临界区，显式检查 copy_from_user、kstrtol 的返回值。Rust 使用原子变量和 Result 解析；这些实现选择有助于比较风险，但 C 也支持原子操作，Rust 也能使用锁，不能把差异全部归因于语言。

write! 通过格式化 trait 约束类型，自定义 writer 检查缓冲区容量；FFI 裸指针有效性仍需审查。C 的格式化调用也可能受到编译器检查。非法输入是运行时走错误分支，不是编译器看到输入后报错。

讨论 逐一指出用户指针在哪里拷贝、哪一步完成 UTF-8 校验、哪一步修改计数。Ordering::Relaxed 仅保证各原子操作自身的原子性，不保证多个统计字段组成同一时刻的快照。

5 实验三 proc 命令计数器

目标 对应 PPT 第 24 页。在数值读写基础上识别 inc、dec、reset 和整数；记录 write_count、command_count、invalid_count，并解释失败分支。

```
cd "$LAB/07-rust-proc-command"
make clean && make
sudo insmod rust_proc_command.ko
P=/proc/rust_lab_cmd_counter
cat "$P"
printf '12\n'    | sudo tee "$P" > /dev/null
printf 'inc\n'   | sudo tee "$P" > /dev/null
printf 'dec\n'   | sudo tee "$P" > /dev/null
printf 'reset\n' | sudo tee "$P" > /dev/null
printf '%s\n' '-7' | sudo tee "$P" > /dev/null
printf 'bad\n'   | sudo tee "$P" > /dev/null
cat "$P"
```

顺序	输入	counter	write_count	command_count	invalid_count
初始	无	0	0	0	0
1	12	12	1	1	0
2	inc	13	2	2	0
3	dec	12	3	3	0
4	reset	0	4	4	0
5	-7	-7	5	5	0
6	bad	-7	6	5	1

以上统计以新加载模块、无其他写入为前提。短输入进入核心处理后，write_count 记录写入尝试，command_count 只记录成功操作；reset 仅重置 counter，不清空其他统计。

C 版复现 进入 03-c-proc-command，执行 make LLVM=1 clean && make LLVM=1，加载 c_proc_command.ko，将 P 改为 /proc/os_lab_cmd_counter 后重复相同输入。完成后分别卸载 c_proc_command 与 rust_proc_command。

验收 六次写入后 counter=-7、write_count=6、command_count=5、invalid_count=1。read_count 不用于本表的固定数值验收，原因见下一节。

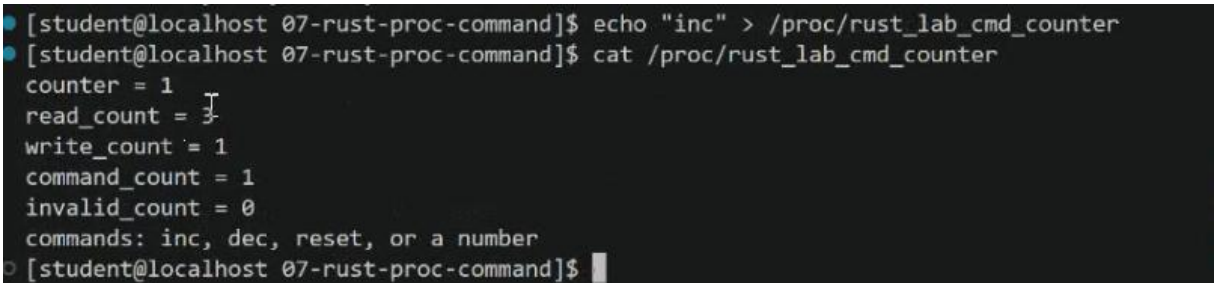


图 8 inc 后 counter 变为 1 截取自实验 3 演示视频约 15 秒处

6 实验三源码分析与并发补充

```
if input == "inc" {
    COUNTER.fetch_add(1, Ordering::Relaxed);
} else if input == "dec" {
    COUNTER.fetch_sub(1, Ordering::Relaxed);
} else if input == "reset" {
    COUNTER.store(0, Ordering::Relaxed);
} else if let Ok(value) = input.parse::<i64>() {
    COUNTER.store(value, Ordering::Relaxed);
} else {
    INVALID_COUNT.fetch_add(1, Ordering::Relaxed);
    pr_info!("rust_proc_command: invalid input\n");
    return -EINVAL;
}
```

图 9 命令分发与 if let 数值解析 对应 PPT 第 24 页

源码对应 rust_proc_command_core.rs 中 rust_cmd_counter_store。inc/dec 用 fetch_add/fetch_sub，reset 用 store(0)，数字分支通过 if let Ok(value) 完成解析与取值；失败增加 INVALID_COUNT 并返回 -EINVAL，成功路径最后增加 COMMAND_COUNT。

并发增量验证

重新加载 Rust 命令模块，先写入 reset，再用两个后台任务各写入 100 次 inc。两路写入全部结束后再读取；此时不混入 set、dec、reset 等其他操作。

```
P=/proc/rust_lab_cmd_counter
printf 'reset\n' | sudo tee "$P" > /dev/null
sudo sh -c '
  for i in $(seq 1 100); do printf "inc\n" > "$1"; done &
  for i in $(seq 1 100); do printf "inc\n" > "$1"; done &
  wait
' sh "$P"
cat "$P"
```

预期 counter=200；相对基线，成功命令数增加 200。以 counter 的最终值验证不会丢失这两路加法，不据此推导 Rust 一定更快，也不把多字段统计看成事务。

读写统计的实现限制

本组 Rust proc_shim.c 的读回调未先判断 *ppos > 0，每次 read 都先调用 render；cat 为确认 EOF 可能再读一次，因此一次 cat 可能使 read_count 增加多次。应结合回调实际进入次数解释。

本组 Rust 桥接写回调最多取前 127 字节，并在成功时返回原始 count；长输入可能被截断。实验采用短输入验证业务。若开展边界扩展，应记录截断行为，而不能把它当作“所有超长输入都返回 EINVAL”。

讨论 为什么实验二的 write_count 不计失败解析，而实验三计入？为什么 reset 不归零累计统计？这些语义由源码决定，应在实验报告中明确。

7 实验四字符设备接口

目标 对应 PPT 第 25 页。由 /proc 转到 /dev，观察 open/read/write/release 的生命周期。设备由 miscdevice 注册，模拟字符设备交互，不要求连接真实打印机。

C 版保存消息

```
cd "$LAB/04-c-char-device"
make LLVM=1 clean && make LLVM=1
sudo insmod c_char_device.ko
ls -l /dev/os_lab_char
cat /dev/os_lab_char
printf 'hello printer\n' | sudo tee /dev/os_lab_char > /dev/null
cat /dev/os_lab_char
sudo rmmod c_char_device
```

预期 第一次显示 message = hello from os_lab_char；写入后 message = hello printer，write_count 增加。C 版写入保存字符串，不把 inc 当作加法命令。关闭统计字段名为 close_count。

Rust 版解析计数命令

```
cd "$LAB/08-rust-char-device"
make clean && make
sudo insmod rust_char_device.ko
ls -l /dev/rust_lab_char
cat /dev/rust_lab_char
printf '10\n' | sudo tee /dev/rust_lab_char > /dev/null
printf 'inc\n' | sudo tee /dev/rust_lab_char > /dev/null
cat /dev/rust_lab_char
printf 'hello\n' | sudo tee /dev/rust_lab_char > /dev/null
cat /dev/rust_lab_char
```

预期 counter 从 0 变为 10，再变为 11。hello 返回错误，counter 保持 11，invalid_count 增加 1；write_count 包含这三次进入核心的写入尝试。关闭统计字段名为 release_count。

源码对应 C 的 os_lab_fops 与 os_lab_misc_device；Rust 的 rust_char_opened、rust_char_render、rust_char_store、rust_char_released，以及 char_shim.c 的 lab_fops。

验收 保存 /dev 节点、两种写入后的状态和错误提示。先保留 Rust 模块以完成下一节的现象分析；清理时卸载 rust_char_device，确认设备节点消失。



图 10 实验 4 视频约 45 秒处 counter 为 888 open 为 4 release 为 3

8 字符设备计数现象与资源清理

现象 对应 PPT 第 26 页。在无并发、无其他持有句柄的顺序实验中，cat /dev/rust_lab_char 读取到的 release_count 通常比 open_count 少 1。原因是当前 cat 已打开设备，但其关闭动作尚未发生。

阶段	回调与事件	计数变化
打开	open → lab_open → rust_char_opened	open_count 增加 1
读取	read → lab_read → rust_char_render	read_count 增加；格式化当前 open/release
返回显示	结果复制回用户缓冲区，由 cat 输出	返回的数据反映本次关闭前的状态
关闭	关闭最后引用后触发 lab_release	release_count 增加 1

```
# 使用新加载的 Rust 字符设备模块，且没有其他访问者
cat /dev/rust_lab_char
sudo dmesg | tail -n 20
cat /dev/rust_lab_char
sudo dmesg | tail -n 20
```

结合 opened/released 日志检查调用顺序。下一次 cat 又先新增一次 open，因此仍可能在输出中看到差 1。这是观测时刻的结果；若同时有其他程序打开设备，差值可能大于 1。release 与文件对象最后一个引用关闭相关，不等于任何场景下每次 close 都各触发一次。

关键源码

源码 08-rust-char-device/rust_char_device_core.rs 第 63 至 73 行

```
#[no_mangle]
pub extern "C" fn rust_char_opened() {
    OPEN_COUNT.fetch_add(1, Ordering::Relaxed);
    pr_info!("rust_char_device: opened\n");
}

#[no_mangle]
pub extern "C" fn rust_char_released() {
    RELEASE_COUNT.fetch_add(1, Ordering::Relaxed);
    pr_info!("rust_char_device: released\n");
}
```

char_shim.c 通过 .owner = THIS_MODULE 维护打开设备时的模块引用。文件仍被占用时 rmmod 可能返回 Module is in use；关闭句柄后再卸载，不能强制卸载。

清理路径 C 的 __exit 调用 misc_deregister；Rust 的 Drop 调用桥接注销函数，再由 C 层执行 misc_deregister。Drop 能将卸载清理绑定到模块对象生命周期，但注销逻辑及 FFI 契约仍须正确。

边界提醒 Rust 字符设备桥接也截取至多 127 字节。自定义 BufferWriter 含 unsafe 指针拷贝，且调用处忽略格式化错误；业务验证不等同于已经证明整个驱动不存在内存或截断问题。

9 补充实验 模块并存与运行期回调

目标 对应 PPT 第 26 页。验证两个模块可以同时登记在运行内核中，并观察独立模块的卸载不会使另一个模块的接口消失。开始前卸载上一组的 rust_char_device，以新加载状态执行。

```
sudo insmod "$LAB/05-rust-hello/rust_hello.ko"
sudo insmod "$LAB/08-rust-char-device/rust_char_device.ko"
lsmod | grep -E '^(rust_hello|rust_char_device) '
cat /proc/modules | grep -E '^(rust_hello|rust_char_device) '
cat /dev/rust_lab_char
sudo rmmod rust_hello
lsmod | grep -E '^(rust_hello|rust_char_device) '
cat /dev/rust_lab_char
sudo rmmod rust_char_device
lsmod | grep -E '^(rust_hello|rust_char_device) '
test ! -e /dev/rust_lab_char && echo device_removed
```

观测时刻	预期模块状态	设备接口
加载二者后	同时有 rust_hello 与 rust_char_device	/dev/rust_lab_char 可读
卸载 Hello 后	仅剩 rust_char_device	继续可读
卸载字符设备后	二者均不在列表	节点消失

可得结论与证据边界

lsmod 与 /proc/modules 的结果说明内核维护并对外提供模块列表，可以同时登记多个模块。该实验与模块登记和移除机制一致，但不能单凭列表输出证明内部链表的全部实现细节。

两个模块之间没有直接的导出符号调用，因而不能通过此实验验证“依赖模块缺失导致 Unknown symbol”。若专门验证符号依赖，需要额外构造导出/导入符号的模块对，并使用 modinfo、nm 和 dmesg 记录依赖；这属于扩展任务。

联系运行期分发 加载时解析和重定位使函数地址可用；运行时 cat 的 read 通过 VFS 与操作表调用已注册函数，不会为每次 read 再读取 Module.symvers。

验收 保存三次模块列表和设备读取结果，解释为什么卸载 Hello 不影响字符设备。不得卸载系统磁盘、网卡或文件系统模块来代替此验证。

10 常见问题与排查顺序

问题	检查重点	处理方法
SSH 连接失败	VMware 网络、当前 IP、sshd、网卡状态	ip -br addr；systemctl status sshd；检查 NetworkManager 连接
Rust 选项不可选	工具版本、rust-src、bindgen、libclang、配置依赖	在源码树运行 make LLVM=1 rustavailable，逐项修复
No rule to make target proc_shim.o	06、07 中是否有 proc_shim.c	从原编译产物目录补齐同组桥接文件
char_shim.o 缺失	08 中是否有 char_shim.c	补齐桥接文件后重新构建
Invalid module format	dmesg、uname -r、vermagic、构建配置与符号版本	指向当前运行内核的完整构建树，清理后使用匹配工具链重编
Unknown symbol	依赖模块、导出符号、符号版本与构建产物	查看 dmesg 中具体符号，确认依赖和 Module.symvers 来源
Key was rejected / 签名失败	Secure Boot 与签名强制策略	按实验虚拟机策略配置模块签名或受控测试环境
File exists	同名模块是否已加载	先 lsmod 检查并正常卸载，再加载新产物
Permission denied	节点权限、写入端是否提权	使用 printf … | sudo tee 路径；检查日志读取权限
Module is in use	设备是否被进程打开	用 fuser/lsof 查占用，关闭句柄后正常卸载

建议保留的排障命令

```
uname -r
cat /proc/version
readlink -f /lib/modules/$(uname -r)/build
modinfo -F vermagic ./模块名.ko
sudo dmesg | tail -n 40
lsmod | grep 模块名
```

注意 vermagic 主要包含内核 release 与若干配置特征，并不是完整的编译器版本清单。重启本身不会必然导致模块失效，应检查是否启动了不同内核、改变了构建树或沿用了旧 .ko。

PPT 对照索引

第 3 至 4 页：环境搭建与工具链核验。第 6 至 11 页：内核模块架构、动态加载、符号与 VFS。第 16 至 19 页：Hello。第 20 至 23 页：数值计数器。第 24 页：命令解析。第 25 至 26 页：字符设备、计数现象及模块并存。

学习资料

1 第六组汇报.pptx，第 3 至 26 页。插图取自对应页面的原始图片。

2 本项目“源代码干净”目录及“源代码及其编译产物”中的桥接源码与构建记录。

3 （翻转课堂）任务指导书模板.pdf，采用其“实验环境构建、实验设计、实验流程”的章节结构。

4 “视频展示全流程”目录中的实验 1、实验 2、实验 3、实验 4-2x 演示视频；终端插图按各图说明的时间截取。

5 Linux 内核构建与 Rust 文档：https://docs.kernel.org/kbuild/modules.html；https://docs.kernel.org/rust/quick-start.html。旧内核的具体要求以对应源码树 Documentation/rust/ 与 rustavailable 检测为准。

6 openEuler 文档：https://docs.openeuler.org/；Rust 工具链安装：https://rustup.rs/。